Claude Certified Developer

Foundations - Exam-Oriented Study Notes

Concepts, decision tables, keyword guide and quick-review cards

Unofficial community notes. Not affiliated with, endorsed by, or produced by Anthropic. This is not an official question bank, answer key, or exam content, and it contains no exam questions. It is a personal synthesis of general concepts from public documentation and hands-on practice. Product behavior, model names, parameters and limits change: **always verify against the official Anthropic documentation** before relying on any detail.

How to study with these notes

Read the decision tables first. Then use the keyword hints to recall the concept and produce your own answer before looking at the solution. Finally, check the traps and version notes.

Order: topic guide, 30 quick-review cards, sources, glossary.

On any scenario question, do not pick the answer that matches a keyword. Pick the one that satisfies the **deciding constraint** in the scenario: security, format, correctness, cost, time, or context.

Contents

1. Study priorities and how to read scenario questions
2. API, retries and operations
3. Streaming and the tool-result contract
4. Prompt caching, batch and cost
5. Context, memory and long-running tasks
6. Agents, workflows and subagents
7. Prompt design and real correctness
8. Claude Code: instructions, skills, plugins, hooks
9. MCP, deployment and security
10. Model and version traps
11. 30 quick-reference cards
12. Official sources and verification notes
13. Glossary

1. Study priorities and reading the question

Scenario questions repeatedly test a small set of distinctions. The table below lists the ones worth drilling first. It is a qualitative study order, not a scored domain breakdown.

Priority	Distinction to practice	Ask yourself
P1	Structured output vs. semantic validation	Is the JSON well-formed, or is the content actually correct?
P1	PreToolUse vs. PostToolUse	Do I need to block an action, or observe its result?
P1	SSE fragments vs. a completed block	Is this a partial piece, or a complete executable tool call?
P1	Cache vs. compaction vs. memory	Is the problem cost, context size, or durable knowledge?
P2	Batch vs. bounded concurrency	Is next-day fine, or is there a hard two-hour deadline?
P2	Workflow vs. agent vs. workers	Are the steps fixed, or does the model choose the next step?
P2	Model and platform compatibility	Is this parameter supported on this model and surface?
P2	Eval, trace, release control	Am I measuring real outcomes, or only the final answer?

Decision method

Constraint in the scenario	Correct solution layer
"must never", "deterministically", "before execution"	Harness, permissions, backend authorization
"schema enforced by the API"	Structured outputs / strict tool use
"valid JSON but wrong"	Semantic / domain validation, source checks
"same prefix, input cost"	Prompt caching
"context full, stale tool results"	Context editing, pruning, compaction
"restore after a crash"	Checkpoint + durable state + idempotency
"works in playground, fails in app"	Request assembly, parser, stream handling, environment

Core rules. A prompt is guidance, not a replacement for a mandatory security control. A stronger model does not fix a wrong tool protocol. Caching does not shrink the context.

2. API, retries and operations

- The official SDK reduces the burden of headers, typed responses, timeouts and retries. Do not stack a blind retry wrapper on top of it; cap the total number of attempts in exactly one place.
- For transient 429/5xx errors, use bounded exponential backoff and honor retry-after when present. SDKs retry transient errors a small number of times by default.
- A 400 (malformed request or input too long) is not fixed by resending. Fix the request shape, token count, or incompatible parameter.
- A 429 is not only about requests per minute. Tokens per minute, sudden traffic spikes (acceleration limits) and spend limits can also cause it. Do not assume every 429 clears after a short wait.
- On every call, track model, prompt version, request ID, usage, latency and error type. Send the individual request ID to support.

Keyword	Choose	Avoid
"SDK already retries", "traffic amplified"	Merge retry layers / set max_retries	Immediate retry loops
"429 at low RPM, huge documents"	Inspect token limits	Only raising the RPM limit
"529, feed must stay available"	Backoff plus a safe fallback	Infinite retries
"400 unsupported field"	Fix model/platform/beta compatibility	Resending the same JSON
"monthly spend by feature"	Usage data plus your own feature tags	Counting characters

Auth note. Never put secrets in a URL or a prompt. Check the current docs for the authentication header your surface expects (the API has used both an x-api-key header and bearer-style auth). An anthropic-version header and a JSON content type are required on the raw API; cloud-provider surfaces use their own IAM methods.

3. Streaming and the tool-result contract

Streaming: what arrives and how to handle it

- Messages streaming uses SSE over HTTP. It is not a bidirectional chat protocol; you cannot send new user input over the same response channel.
- Typical flow: message_start, then indexed content blocks, then message_delta and message_stop. Ping, error and new event types can appear in between.
- Text arrives as text_delta, tool arguments as input_json_delta.partial_json, and thinking as its own delta type. Do not append everything to one text buffer.
- Accumulate partial_json fragments per block and parse only after content_block_stop (or use an SDK accumulation helper).
- Usage in message_delta is cumulative: do not sum every delta. The stop reason also arrives in that event.

Exam sentence: "Every fragment fails JSON.parse, but the final arguments are valid" means accumulate the fragments and parse when the block completes.

Truncated streams. Never replay or execute a half-finished tool or thinking block as if it were complete. For text, use a supported continuation pattern; for tool calls, obtain the complete result again. Streaming reduces connection risk but does not eliminate every timeout or error.

Tool loop invariants

Situation	Correct behavior
Model returned tool_use	The harness executes client tools
Three parallel calls arrived	A result for every ID, all tool_result blocks in the same next user message
A tool timed out	Return an error result with the matching ID; never ignore the call
A batch tool partially failed	Report per-ID partial success/failure
Tool result is an image	Use an image block inside tool_result.content, not base64 as plain text
Thinking is on and results go back	Preserve assistant blocks completely and unmodified

Stop reasons

Value	Meaning / action
end_turn	Normal end; still validate content against acceptance criteria
tool_use	Send client tool results; a server tool call may also be pending
pause_turn	A server-tool loop paused; keep the assistant content and continue
max_tokens	Output may be cut off; adjust the budget or the task
model_context_window_exceeded	Stopped at the context limit; create headroom
refusal	Safety refusal; do not blindly retry as if it were a transient error
stop_sequence	A defined delimiter was produced; read which one

Mixed tools trap. A server tool does not always finish in the first response. Send client results in the correct tool_result message, keep the same tool definitions, and never fabricate a server result in your application.

4. Prompt caching, batch and cost

Caching

- The cache covers the whole prefix in order: tools, then system, then messages. You cannot cache system while skipping the tools that come before it.
- Put static instructions first and variable user content after. Place the breakpoint at the end of the section that stays the same.
- Default TTL is 5 minutes; consider the 1-hour TTL for reuse every 20 to 40 minutes. Each hit refreshes the lifetime.
- There is a maximum of four breakpoints. Automatic caching advances the marker in a growing conversation; explicit markers separate sections that refresh at different rates.
- Minimum cacheable size depends on model and platform. Do not memorize a token count; measure real cache writes and reads from usage.

Keyword	Choose	Trap
"fixed instructions + hourly catalog"	Separate explicit breakpoint after the instructions	Placing the only marker at the end of the catalog
"growing append-only chat"	Automatic or advancing-breakpoint strategy	Changing the whole prefix every turn
"tools precede system"	Cumulative prefix logic	Skipping tools in the cache
"short unique two-turn chats"	Measure the economics; avoid needless caching	Assuming cache writes are free
"TTL expired between queries"	A suitable TTL	Upsizing the model to fix a cache problem

Worked example. If the question gives a 5-minute write at 1.25x and a read at 0.1x, two uses cost 1.35x versus 2x without caching. With no reuse, the 1.25x write is more expensive. These are standard example ratios; check the pricing table for model-specific exceptions.

Batch or concurrency?

Requirement	Choose
Nightly archive summaries, consumed in the morning, cost first	Message Batches
Thousands of independent jobs, a two-hour deadline, spare capacity	Async SDK + bounded concurrency
User should see the first text immediately	Streaming
Avoid re-uploading the same document	Files API (does not zero out input token cost)

Batch gives a 50% price discount and can take up to 24 hours. Match results by custom_id, keep successes, and selectively retry retryable failures. Documented limits at the time of writing: 100,000 requests or 256 MB per batch; download results within 29 days of creation. If you submitted a wrong template, cancel the still-running batch; do not expect cancellation to undo completed work.

5. Context, memory and long-running tasks

Concept	What it solves	What it does not solve
Prompt caching	Cost/latency of reprocessing a prefix	Context size
Pruning / context editing	Reduces old low-value tool content	Durable job state
Compaction	Continues a conversation via a summary of key decisions	Word-for-word preservation of every detail
Retrieval / RAG	Fetches relevant information from an external store	Having covered an entire archive you did not retrieve
Durable memory / state	Preferences and progress across sessions	The API remembering by itself
Checkpoint + idempotency	Safe continuation after a crash	Making blind repetition of an action safe

- "Old tool results fill the window": prune the old results; keep recent findings, decisions and open items.
- "Repeats completed work after compaction": add a structured persistent task list or checkpoint.

- "Knowledge base far too large, only a small slice relevant": just-in-time retrieval; do not put the whole KB in every request.
- "Every archive report must be reflected": process all the pieces and merge intermediate summaries hierarchically; top-k retrieval alone is not enough.
- "Reason over 600K tokens at once": evaluate a supported large-context model or platform; besides window size, weigh cost, media limits and evals.

Current caution. On newer models, preserved thinking can be tied to the earlier prefix. Hand-editing history can cause signature errors, not just cache misses. Use supported server-side compaction or context editing, or a documented transformation; do not delete random blocks.

6. Agents, workflows and subagents

- **Workflow:** steps and paths are defined in code. First choice for a fixed extract, check, report pipeline.
- **Agent:** the model chooses the next step or tool based on observations. Executing an upfront plan without looking at results throws away the benefit of the agent loop.
- **Routing:** separate prompt, tool or model paths for distinct kinds of work; routine work to a cheaper model, hard cases to a stronger one.
- **Parallelization:** independent parts for speed, or independent review attempts for recall and confidence.
- **Evaluator-optimizer:** generate, evaluate, revise when there is a clear quality criterion and measurable improvement. Not every task needs an extra call.

Keyword	Choose
"subtasks unpredictable; varying searches"	Orchestrator-workers
"same steps every time"	Prompt chaining + intermediate gate
"optimizing escalations hurts resets"	Routing + specialized paths
"standard loop, no unusual flow"	Ready-made plumbing (tool runner / Agent SDK)
"one retrieval, short response"	Simple retrieval + a single model call

Subagent handoff. Give the subagent a goal, the prior decisions it needs, sources, currency/units, tool limits and an output contract. Do not assume it knows the parent conversation. Return a short finding plus source plus uncertainty, not a full transcript. Do not spawn agents for tiny micro-tasks. When independent code tasks collide on shared files, use file ownership or worktrees and a controlled merge.

Termination. Set a success criterion, a maximum number of turns, a total spend cap and a timeout. If a tool timeout is not returned to the model, the model cannot recover. Do not declare success because a file was written; run the tests or the build.

7. Prompt design and real correctness

- Keep persona and standing rules in the system layer; do not mix user data with instructions. The trusted per-session context should be produced by your backend.
- Separate translation or summary text from instructions with explicit delimiters, and say not to follow instructions inside it. This first measure alone does not solve injection.
- Few-shot examples should represent different classes, language/format variation and edge cases. Ten examples from one category can bias the model.
- Replace "short/professional" with measurable length, fields and an example. Pair prohibitions with the expected behavior and the reason.
- If the answer may be absent from the source, define a null/unknown/abstain option, and ensure a fabricated field is never shown as equal to a real one.

Three separate guarantees

Need	Control to choose
JSON shape and fields	Structured outputs (output_config.format)

Need	Control to choose
Tool input shape	strict: true + a supported schema
Is the content correct?	Source / domain / semantic validation
Is the action authorized?	Backend authorization
Is the math right?	Deterministic computation (code)

Exam sentence: "HTTP 200 + valid JSON + wrong figures" means semantic validation, not a stricter JSON prompt. Refusals and max-token truncation can break a structured-output schema, so keep checking the stop reason.

Eval tactics. Use a representative frozen regression set plus separate tuning and held-out examples. Mock or stub in unit tests, and add a few small controlled live integration tests. For open-ended text, prefer rubric or semantic criteria over exact-string matching; for code, verify build, tests and outcomes. Calibrate any LLM judge against human-labeled samples.

8. Claude Code: instructions, skills, plugins, hooks

Requirement	Mechanism
Every session should know project rules	CLAUDE.md
Personal instructions across all repos	~/.claude/CLAUDE.md
Long procedure, only when relevant to the task	Skill
One command starts a release procedure	Slash-invoked skill / command
Separate read-only specialist	Custom subagent + tool allowlist
Distribute skill, command and hook together	Plugin
Team permission / hook configuration	.claude/settings.json
Personal repo settings, not committed	.claude/settings.local.json
Nobody can loosen it locally	Administrator-managed settings

Changes to repo instructions and hooks should go through PR review like code. Plugins should also declare their dependencies and environment; "a single package" does not mean every external service and credential is inside it. A skill supplies procedural knowledge; MCP provides access to executable external capability.

Choosing a hook

Keyword	Choose	Why
"block before run"	Command-based PreToolUse	Decision before execution
"ask engineer on match"	permissionDecision: ask	Human decision
"never approve in the moment"	A deny control	Ask can be approved; deny cannot
"log every successful tool result"	Observe-only PostToolUse	Monitors results
"also failures"	Cover PostToolUseFailure	Do not miss failures
"formatter/test after edit"	PostToolUse	Post-hoc automation
"warning printed but it ran"	Correct deny JSON, or exit 2 on a block-capable event	A normal exit is not the same as blocking

Notes. An immutable audit log comes from log-store access, retention and WORM design, not from appending to a file. A post hook cannot undo an action. A model or prompt hook can give deterministic timing, but an LLM decision is not deterministic. **Version trap:** older material lists allow/deny/ask for permissionDecision; current docs may also show a defer option. Check the official hooks reference for the exact value list and headless vs. interactive behavior.

9. MCP, deployment and security

English hint	Concept
"model-callable action"	Tool
"attach read-only document as context"	Resource
"user menu, region/quarter parameters"	Prompt
"server asks client for generation, no key in server"	Sampling
"same tools, non-Claude client"	Open, provider-independent protocol
"local subprocess"	stdio (diagnostic logs go to stderr)
"central remote server"	Streamable HTTP + authentication

The host application contains the client; the client connects to servers; servers expose capabilities. Treat community server descriptions and results as untrusted input: review code and dependencies, pin versions, grant narrow permissions, and update deliberately.

Pick the right layer for security

- **Auth is not authorization.** Signing in with SSO does not mean the user may access every record. The tool/backend applies tenant and record filters based on the authenticated identity.
- **Audit is not prevention.** Logging a deletion does not limit who can delete. Use least privilege and approval before the action.
- **A schema is not safe SQL.** Validate parameters in the application, use parameterized queries, and prefer a narrow task tool or read-only role over free-form SQL with admin credentials.
- **Secrets do not belong in model context.** Keep API keys out of prompts, repos and mobile bundles. If a key is exposed, revoke and rotate it; a git revert alone is not enough.
- **Untrusted content is not instructions.** Combine a data boundary, narrow tool permissions, backend policy, sandboxing and approval for high-impact actions. If PII must not leave your environment, redact or tokenize before sending.

Deployment tactics. For "internal DB behind a firewall", think about where tools execute. For "state must not be stored outside the company", separate history/control plane from data retention. An agent harness running locally does not mean inference is local. Even in a managed self-hosted sandbox, verify orchestration and state conditions separately. These notes do not guarantee compliance; check the official deployment and retention pages.

10. Model and version traps

Old habit	Current exam approach
"For JSON, always prefill with {"	Prefill is rejected on newer models; use supported structured outputs
"Thinking always uses budget_tokens"	Legacy manual on older models; deprecated on later ones; adaptive on newest
"Thinking and forced tool use never combine"	Incompatible in manual mode; adaptive mode has model-specific exceptions
"strict: true makes every turn call a tool"	Schema conformance is not the same as forcing a call
"Temperature 0 guarantees identical repeats"	Not a determinism guarantee; use semantic evals
"No date suffix means a fixed alias"	Verify canonical vs. fixed IDs via the Models API
"Every context overflow is a 400"	Newer models may return a stop reason at the output context limit
"Every beta example needs a header"	Check per feature, version and platform
"Every cache hit costs 0.1x"	Some models have pricing exceptions
"Bedrock always uses the legacy body/event format"	Distinguish the current Messages endpoint from the legacy runtime

The official Errors, Thinking and Release Notes pages are the primary references for these changes. If a question names a specific model, apply that model's rules, not the newest model's. Some newer models reject

manual enabled/budget_tokens (use adaptive thinking with an effort setting), may reject non-default sampling controls, and may reject forced tool choice (any/tool), so "guaranteed tool call on every model" is wrong. Verify the exact model list in the docs.

11. 30 quick-reference cards

Card	Definition	Keywords	Trap	Pick
01 Retry/backoff	Bounded waiting retries on transient errors	429, 529, transient, retry-after	Retrying a 400; a second loop on top of the SDK	One retry policy + bounded backoff
02 Request ID	Identifier that lets support find one API request	support cannot locate, identifying header	Confusing a session ID with a request ID	Log the response request-id and send it
03 SSE	Incremental event stream over an HTTP response	token by token, SDK vs REST	Assuming WebSocket / duplex is required	stream=true + official SDK streaming helper
04 Streaming assembly	Combine deltas by content index and type	partial_json, mixed prose and tools	JSON-parsing every fragment	Accumulate the full block, parse on completion
05 Stop reason	Shows why generation ended	max_tokens, pause_turn, refusal	Treating every HTTP 200 as complete success	Separate dispatch path per stop reason
06 Tool-result contract	Result blocks matched by call ID	unanswered invocation, parallel tools	Plain user text, or matching by order	A matching tool_result for every call
07 Strict tool schema	Constrains tool input with a supported schema	enum, exact codes, invalid arguments	Thinking a schema provides authorization	strict + enum + backend validation
08 Structured outputs	Constrains the response JSON schema via the API	API enforced, missing fields, prose preamble	Equating valid format with correct information	output_config.format + check stop reason
09 Semantic validation	Verify meaning against source or business rules	valid JSON but wrong, silent failure	Only running a parser	Grounding, domain checks, escalation where needed
10 Deterministic computation	Compute totals/formulas with real code	right extracted figures, wrong arithmetic	Treating LLM confidence as proof	Code or a tool does the calculation
11 Prompt caching	Cuts the cost of reprocessing the same prefix	stable prefix, repeated input	Believing it shrinks context	Cache breakpoint at the end of the static prefix
12 Cache TTL	Window during which an entry can be reused	20 minutes, expires, repeated writes	Expecting a hit on a 5m TTL	Suitable 1h TTL; measure with usage
13 Message Batches	Bulk-process independent, latency-tolerant jobs	overnight, archived, next morning	Assuming a two-hour deadline guarantee	Batch + polling + results archive
14 Bounded concurrency	Controlled parallelism for independent live calls	sequential slow, unused capacity, two hours	Unbounded fan-out	Async SDK, semaphore/worker pool, rate pacing
15 Compaction	Keeps a high-value summary of the old conversation	long sessions, preserve decisions	Assuming every detail is preserved losslessly	Decision/work summary + recent turns + state
16 Retrieval/RAG	Fetches relevant external information per question	huge KB, small slice	Stuffing every document into the prompt	Just-in-time relevant chunks + source references
17 Durable state	Preferences and progress that persist across sessions	remember next session, repeats work	Using CLAUDE.md as an app database	Scoped durable store + retrieval when needed
18 Checkpoint/idempotency	Safe continuation of a half-finished task	restart, duplicate migrations	Assuming a restart from scratch is safe	Durable checkpoint + IDs that prevent repeated effects

Card	Definition	Keywords	Trap	Pick
19 Workflow	A predefined sequence of steps	steps never vary, fixed pipeline	Making every fixed job an agent	Prompt chaining + intermediate programmatic gates
20 Routing	Sends work to a specialized prompt/model/tool path	routine cheap, complex minority	Sending everything to the priciest model	Measured classifier/routing + escalation
21 Orchestrator-workers	Dynamic subtasks with central synthesis	unpredictable subtasks, parallel research	Duplicate workers with the same generic prompt	Distinct scope, ownership, short output contract
22 Subagent handoff	Explicit transfer of needed context to a subagent	wrong currency, fresh phase contradicts	Assuming parent history is visible	Put decisions, sources, units, acceptance criteria in the task
23 Evaluator-optimizer	Iterative improvement with feedback	clear rubric, refinement valuable	Requiring a second LLM for every job	Bounded critique loop when benefit is measurable
24 CLAUDE.md	Project guidance applied in a session	always-on conventions, onboarding	Treating it as deterministic security or runtime state	Short shared guide, version-controlled and reviewed
25 Skill	Procedure and resource bundle loaded on demand	on demand, reusable checklist	Embedding every procedure in a giant system prompt	Task-specific SKILL.md with a good description
26 Plugin	Distributes several customizations together	commands + hooks + skill, installable	Assuming external dependencies install themselves	Versioned package + documented/provisioned dependencies
27 PreToolUse	Programmable control before execution	block before, no bypass by model	Printing a warning with exit 0 and expecting a block	Command deny decision; backed by tests and a sandbox
28 PostToolUse	Automation after a successful tool result	audit every result, after edit	Trying to stop an irreversible action afterwards	Observe-only logging or test/formatter; add failure coverage
29 Least privilege	Only the access the task needs	unauthorized deletion, admin credentials	Treating logs and prompts as authorization	Narrow backend role + identity/tenant validation + approval
30 MCP	Common protocol for tools, resources and prompts	non-Claude client, user menu, remote server	Mixing up resource, prompt and tool roles	The right primitive for the need + secure transport

12. Official sources and verification notes

Living documentation changes without notice. Treat this list as where to verify, not as proof that every detail above is current.

Source	Use for
API Overview	Auth, SDKs, platform differences
API Errors	Retry behavior and request IDs
Streaming	Events, partial JSON, recovery
Stop Reasons	Tool / pause / refusal / truncation
Prompt Caching	Prefix, TTL, minimums, usage
Batch Processing	Limits, custom_id, result windows
Structured Outputs	JSON format vs. strict tool use
Thinking	Adaptive thinking, effort, blocks to preserve
Release Notes	Dates of version changes
Building Effective Agents (Anthropic, Dec 2024)	Architecture decisions

Source	Use for
Demystifying Evals for AI Agents (Anthropic, Jan 2026)	Graders, traces, tests
Claude Code Hooks reference	Exact hook schema and value lists

13. Glossary

Term	Short meaning	Term	Short meaning
API	Programmatic interface to a service	Prefix	The shared beginning of a prompt
SDK	Client library for an API	Cache hit / miss / write	Reused / not found / entry created
Endpoint	The address a request is sent to	Breakpoint	Marks where the cacheable prefix ends
Header	Request/response metadata	TTL	How long a cache entry lives
Authentication	Verifying who the caller is	Token	Unit of text a model processes
Authorization	Deciding what the caller may do	Context window	Maximum tokens a model can consider
Bearer token	Credential sent in a header	Compaction	Summarizing history to free context
Secret manager	Store for credentials	Pruning	Removing low-value content
Key rotation	Replacing a credential on a schedule or after exposure	Retrieval / RAG	Fetching relevant data to ground generation
Revocation	Withdrawing a credential or permission	Embedding	Vector representation of meaning
Tenant	A customer-scoped boundary	Memory / Durable state	Information persisting across sessions
Least privilege	Grant only the access a task needs	Checkpoint	A saved point to resume from
RBAC	Role-based access control	Agent	A model that chooses its own next steps
Rate limit	Traffic quota	Harness	The infrastructure running the agent loop
RPM / TPM	Requests / tokens per minute	Workflow	Predefined sequence of steps
Acceleration limit	Limit on how fast traffic may grow	Orchestrator / Worker	Coordinator / task executor
Retry	Repeating a failed call	Subagent	An isolated specialist agent
Exponential backoff	Increasing waits between retries	Handoff	Transferring task and context
Jitter	Random variation in wait time to avoid synchronized retries	Routing	Directing work to the right path
Timeout	Maximum time allowed for an operation	Parallelization	Running independent work concurrently
Fallback	Alternative path when the primary fails	Concurrency	Overlapping execution
Circuit breaker	Stops calls to a failing dependency	Batch processing	Asynchronous high-volume processing
Idempotency	Repeating an operation has the same effect as doing it once	Polling	Periodically checking status
Request ID	Unique identifier for a single request	Custom ID	Your own identifier to match results
Observability	Visibility into system behavior	Evaluation / Grader / Rubric	Quality measurement / scoring logic / criteria
Trace	Record of the steps in an execution	Regression test	Checks that old behavior still works
Audit log	Record of actions for accountability	Canary release	Gradual rollout to a small share of traffic
Immutable log	A record that cannot be altered after writing	Model pinning	Fixing a specific model version
SSE	Server-sent events, one-way streaming over HTTP	Deprecation	Announced retirement of a feature
Delta	An incremental piece of a streamed response	Migration	Moving to a new version or platform

Term	Short meaning	Term	Short meaning
Content block	A typed piece of message content	Extended / Adaptive thinking	Extra reasoning / dynamically allocated reasoning
Partial JSON	An incomplete JSON fragment	Effort	A setting for how much work the model invests
Stop reason	Why generation ended	Prefill	Seeding the start of a response (legacy)
Truncation	Output cut off before completion	Hook	Event-triggered automation
Refusal	Model declines for safety reasons	PreToolUse / PostToolUse	Before / after a tool runs
Tool use / Tool result	A request to act / the observation of that action	Permission decision	Allow, deny or ask outcome
Tool schema	Input contract for a tool	Managed settings	Admin-enforced configuration
Enum	A fixed set of allowed values	Skill	On-demand procedure package
Strict tool use	Tool inputs conform to the schema	Plugin	Distributable bundle of customizations
Structured outputs	Responses conform to a JSON schema	Headless	Non-interactive execution, such as CI
Constrained decoding	Restricting generation to a valid format	Sandbox	Isolated environment limiting impact
Semantic validation	Checking meaning, not just shape	MCP	Model Context Protocol
Grounding	Tying output to source evidence	Host / Client / Server	App / connector inside it / capability provider
Hallucination	Unsupported or fabricated content	Resource	Read-only context data exposed by a server
Abstention	Choosing not to answer when unsure	Sampling	Server asks the client's model to generate
Citation	Reference to supporting evidence	stdio	Local subprocess transport
System prompt	Base instructions and role	Streamable HTTP	Remote transport over HTTP
Few-shot / Zero-shot	With examples / without examples	OAuth	Delegated authorization protocol
Delimiter	Marker that separates content boundaries	PII	Personally identifiable information
Prompt injection	Attack via instructions hidden in data	Redaction	Masking sensitive data
Untrusted input	Any external content you do not control	Data residency	Where data is stored and processed
Prompt caching	Reusing a processed prompt prefix	ZDR	Zero data retention